

《编译原理》课程设计 实验报告书

班 级： 2017

学 号： 011750230

姓 名： 邓梟

指导老师 李静

课设地点： 实验楼 105

课设时间： 2019 年 12 月

2020 年 1 月 8 日

前言

主要工作：对于 C_minus 语法，使用 Flex、Bison 构造词法分析器、语法分析器；随后生成抽象语法树，并在抽象语法树的基础上构建符号表，使用 LLVM API 生成中间代码 IR，并对生成的 IR 进行优化，并用 LLVM 自带的 lli 指令验证 IR 的正确性。

小组成员：邓梹、蔡月嘯

具体分工：

- (1) 两人共同使用 Flex、Bison 进行词法语法分析、生成语法树 AST
- (2) 邓梹负责从语法树到 LLVM IR 的转化处理工作
- (3) 蔡月嘯负责针对生成的 LLVM IR 编写 Pass 进行优化处理

目录

1	C_minus 语法理解	4
2	C_minus 编译器构造	6
2.1	LLVM 编译架构的优势	6
2.2	C_minus 编译器前端总体结构	7
2.3	词法分析器	7
2.4	抽象语法树设计	9
2.5	生成抽象语法树 (AST)	11
2.6	中间代码 (IR) 生成	12
3	系统主要函数说明	13
3.1	yylex()	14
3.2	TokenType getToken(void);	14
3.3	parse()	14
3.4	util.c 文件中的工具函数	14
3.5	printTab(map<function_name,function_Info> functions)	15
3.6	getIR(TreeNode * tree)	15
3.7	yyerror(char * message)	16
4.	中间代码优化	16
5.	课设心得	16
	[参考资料]	17

1 C_minus 语法理解

1. $\text{program} \rightarrow \text{declaration-list}$

2. $\text{declaration-list} \rightarrow \text{declaration-list declaration} \mid \text{declaration}$

3. $\text{declaration} \rightarrow \text{var-declaration} \mid \text{fun-declaration}$

注: program 是文法的开始符号, 它产生一系列定义, 可以是函数或者变量, 并且函数和变量的定义可以任意穿插。可知, 至少有一个变量或函数的定义。

4. $\text{var-declaration} \rightarrow \text{type-specifier ID}; \mid \text{type-specifier ID}[\text{NUM}]$

注: 变量可以是简单变量或数组变量, 数组索引是 $0 \sim \text{NUM}-1$ 。一个定义变量时, 一个类型标识符后面只能跟一个变量, 即不能有 “int a, b, c;” 这种语句, 且定义变量时不能同时给变量赋值。

5. $\text{type-specifier} \rightarrow \text{int} \mid \text{void}$

注: C_minus 只支持 int 和 void 两种类型, 可知 void 类型一般只用于函数返回值

6. $\text{fun-declaration} \rightarrow \text{type-specifier ID}(\text{params})\text{compound-stmt}$

注: 函数定义语句, 可知 C_minus 语法不支持函数声明, 即函数头后紧跟着函数体。

7. $\text{params} \rightarrow \text{param-list} \mid \text{void}$

8. $\text{param-list} \rightarrow \text{param-list}, \text{param} \mid \text{param}$

9. $\text{param} \rightarrow \text{type-specifier ID} \mid \text{type-specifier ID}[]$

注: 函数的参数列表不能什么都不写, 没有参数时要写上 “void”。

10. $\text{compound-stmt} \rightarrow \{\text{local-declarations statement-list}\}$

注: 复合语句, 相当于用花括号括起来的一段代码。复合语句内可以定义只属于本复合语句的变量, 但所有变量必须定义在其他语句之前。优先使用本复合语句内的变量。

11. $\text{local-declarations} \rightarrow \text{local-declarations var-declaration} \mid \text{empty}$

12. $\text{statement-list} \rightarrow \text{statement-list statement} \mid \text{empty}$

注: 复合语句内可以不定义参数和没有语句, 但不用也不能写上 “empty”。写 bison 文件时, 也不能写上 “empty”, 空着即可。

13. $\text{statement} \rightarrow \text{expression-stmt} \mid \text{compound-stmt} \mid \text{selection-stmt} \mid \text{iteration-stmt} \mid \text{return-stmt}$

注：一条语句可以是一个表达式、复合语句、选择语句 (if_else)、循环语句 (while)、返回语句。

14. $\text{expression-stmt} \rightarrow \text{expression}; \mid ;$

15. $\text{selection-stmt} \rightarrow \text{if}(\text{expression})\text{statement} \mid \text{if}(\text{expression})\text{statement} \text{ else } \text{statement}$

16. $\text{iteration-stmt} \rightarrow \text{while}(\text{expression})\text{statement}$

17. $\text{return-stmt} \rightarrow \text{return}; \mid \text{return expression};$

18. $\text{expression} \rightarrow \text{var}=\text{expression} \mid \text{simple-expression}$

19. $\text{var} \rightarrow \text{ID} \mid \text{ID}[\text{expression}]$

20. $\text{simple-expression} \rightarrow \text{additive-expression} \mid \text{relop additive-expression additive-expression} \mid \text{additive-expression}$

注：表达式语句用于求表达式的值，并赋给一个变量、作为函数的返回值，也可以不赋值给任何变量，即可以是一条没有任何作用的语句。表达式可以是算术表达式、函数调用，且必须以英文分号结束。条件语句和循环语句可以互相嵌套。if 和 while 的条件非 0 则执行其后随的语句。

21. $\text{relop} \rightarrow <= \mid < \mid > \mid >= \mid == \mid !=$

22. $\text{additive-expression} \rightarrow \text{additive-expression addop term} \mid \text{term}$

23. $\text{addop} \rightarrow + \mid -$

24. $\text{term} \rightarrow \text{term mulop factor} \mid \text{factor}$

25. $\text{mulop} \rightarrow * \mid /$

26. $\text{factor} \rightarrow (\text{expression}) \mid \text{var} \mid \text{call} \mid \text{NUM}$

注：语法规规定在一个表达式中，函数调用优先于乘除法，乘除法又优先于加减法。

27. $\text{call} \rightarrow \text{ID}(\text{args})$

28. $\text{args} \rightarrow \text{arg-list} \mid \text{empty}$

29. $\text{arg-list} \rightarrow \text{arg-list}, \text{expression} \mid \text{expression}$

注：函数调用时若没有参数，则留空，不填“empty”。

2 C_minus 编译器构造

2.1 LLVM 编译架构的优势

近十几年来，LLVM 是应用最为广泛的编译器框架。

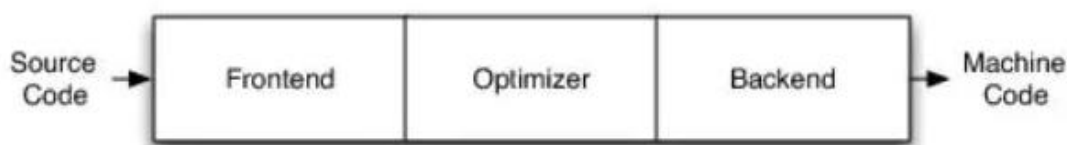


图 1 传统的编译器架构

传统的编译器架构先经过前端的词法分析、语法分析、语义分析、中间代码生成，到中端的中间代码优化，再到后端生成机器代码。它效率不高地主要原因在于，每一种语言都要单独地走完这一流程。为什么我们不生成一种统一的中间代码，再由这种中间代码来转换到不同的机器语言呢？

LLVM 编译架构很好地解决了这一问题。

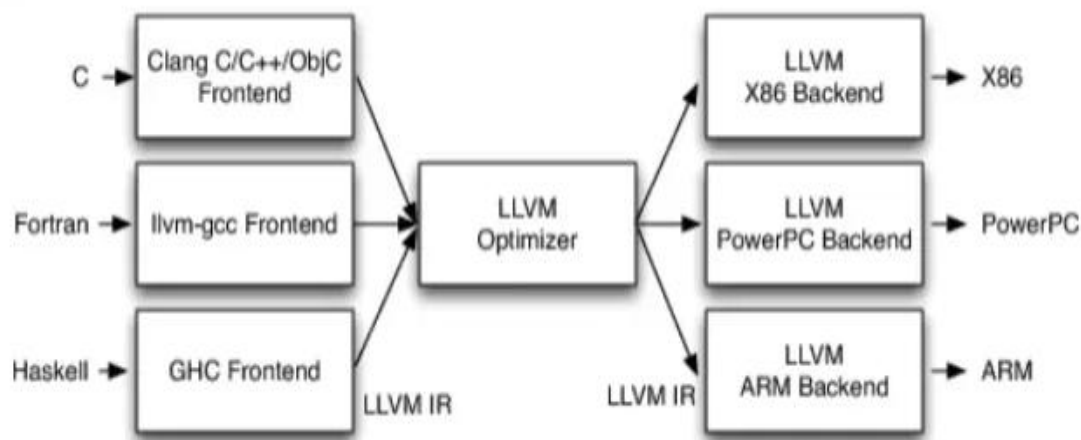


图 2 LLVM 架构

LLVM 使用统一的中间代码（IR），要支持一种新的语言，只要实现新的前端，生成了 IR 之后就可以利用 LLVM 架构生成各种已有后端的指令集的机器代码；要支持新的指令集，只要实现新的后端。LLVM 的中间代码优化阶段是个通用阶段，都是针对 IR 这一种中间代码进行优化，这是 LLVM 优势的主要体现。

相比之下，传统编译器架构前后端界线不明显，要支持一种新语言或支持一种新指令集，就要从头到尾完成编译的全部工作。

2.2 C_minus 编译器前端总体结构

既然 LLVM 优势明显，那么紧随潮流，尝试使用 flex + bison + LLVN 实现 LLVM 架构下的 C_minus 编译器前端，主要流程如下图所示。

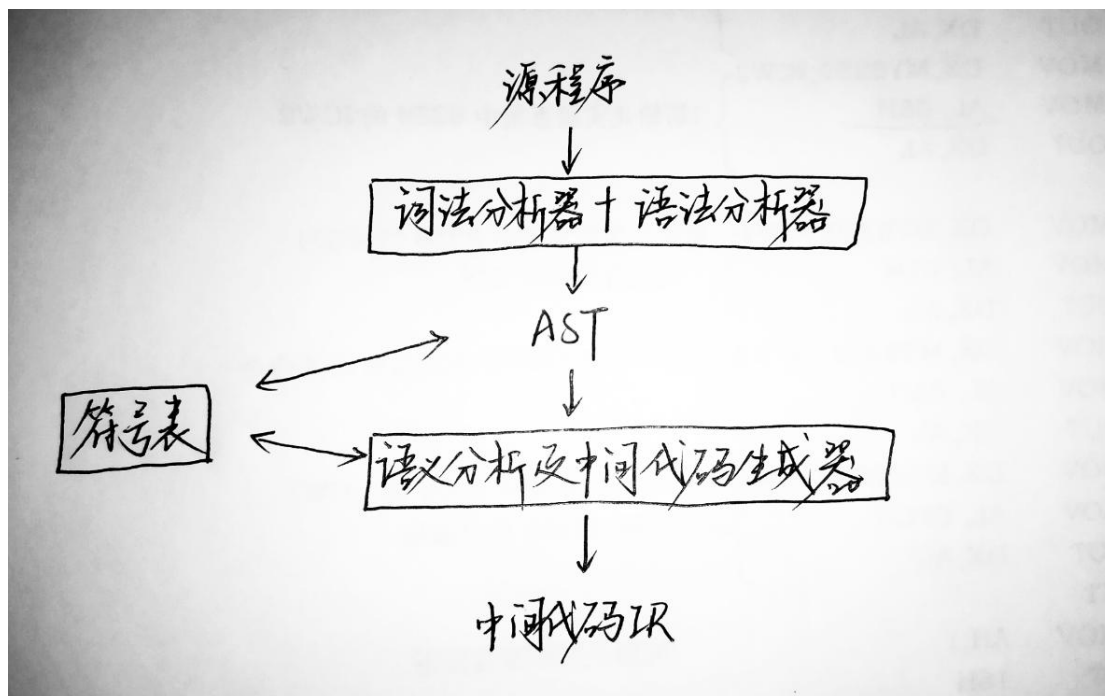


图 3 C_minus 编译器前端

2.3 词法分析器

C_minus 支持的关键字:

void、int、if、else、while、return,

专用符号:

+ - * / < <= > >= == != = ; , () [] { } /* */

标识符和数字常量:

letter = a|..|z|A|..|Z

digit = 0|..|9

ID = letter letter*

NUM = digit digit*

空格由空白、换行符和制表符组成。空格通常被忽略，除了它必须分开 ID、NUM 关键字。注释用通常的 C 语言符号 /* ... */ 围起来。注释可以放在任何空

白出现的位置(即注释不能放在标记内)上, 且可以超过一行。注释不能嵌套。
C_minus 大小写敏感。

词法分析器状态转换图设计:

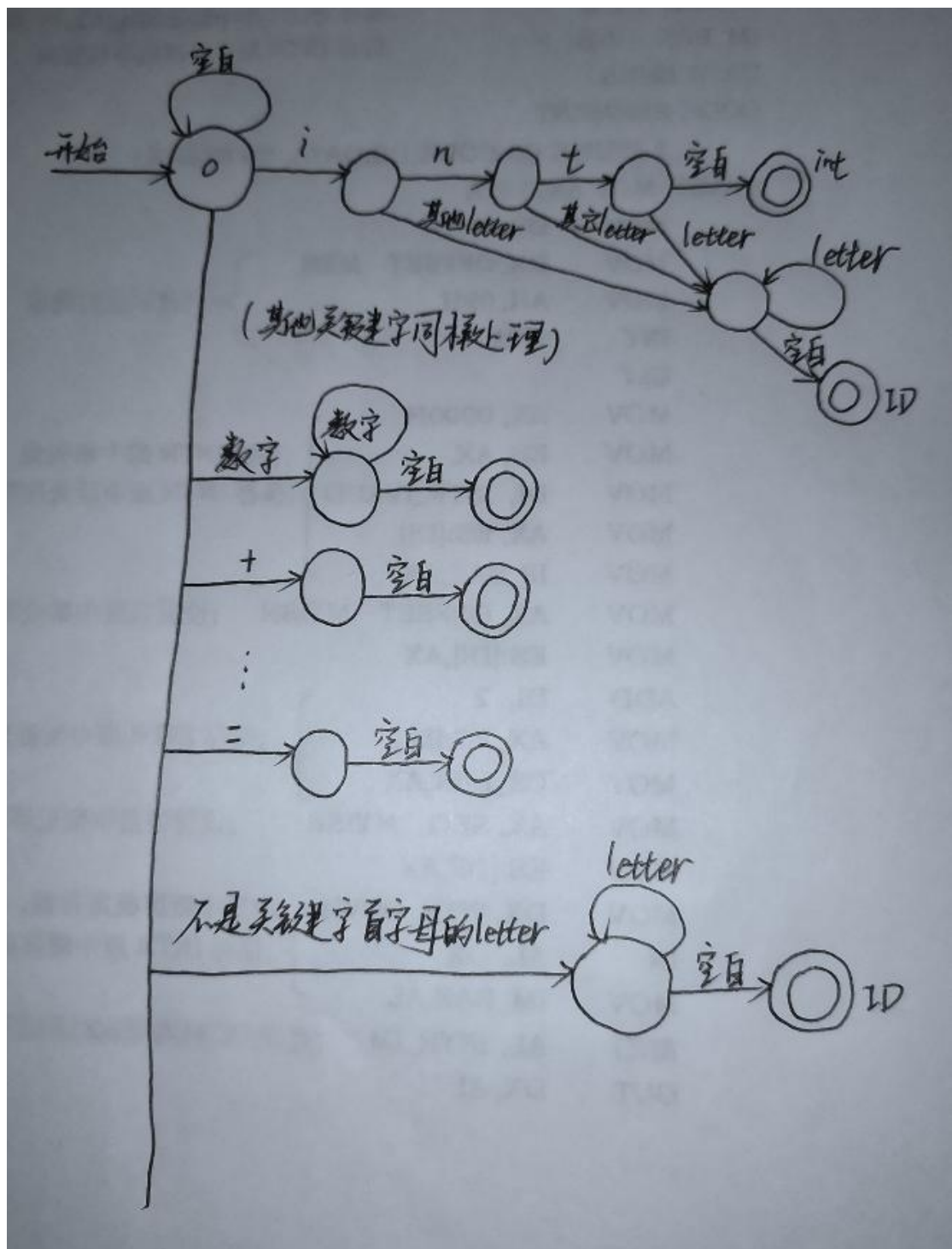


图 4 词法分析状态转换图

没有用代码具体实现词法分析器, 而是用 flex 完成了词法分析。使用 flex 进行语法分析需要注意以下几点:

- ① 因为相同的输入可以被多种不同的模式匹配, 如 "<" 和 "<=", 保留关

键字和用户自定义的标识符。flex 解决词法中的二义性默认有两个原则：一、优先匹配字符多的字符串；二、如果有多个模式可以匹配则优先匹配最先出现的模式。这样，“<=”不会被分析成“<”和“=”，因为“<=”更长。所以在 flex 的.l 文件中，必须先写完所有关键字的匹配模式，再写用户自定义标识符的匹配模式。

- ② 在正则表达式后面不能写注释，否则不能用 flex 运行.l 文件产生 yy.lex.c 文件。

```
digit      [0-9]
number     {digit}+           //这里不能写注释
letter     [a-zA-Z]
identifier {letter}+
newline    \n
whitespace [\t]+
```

- ③ 由于历史原因，当 yylex() 在读取到文件末尾 EOF 时会调用 yywrap()，这是很陈旧的特性，如果不使用它，通过加入 %option noyywrap 关闭。
- ④ 使用 %option noinput 来清除默认输源，即不从控制台输入，而从文件输入，因为从控制台输入一次只能读入一行。

2.4 抽象语法树设计

部分抽象语法树结点示意图：

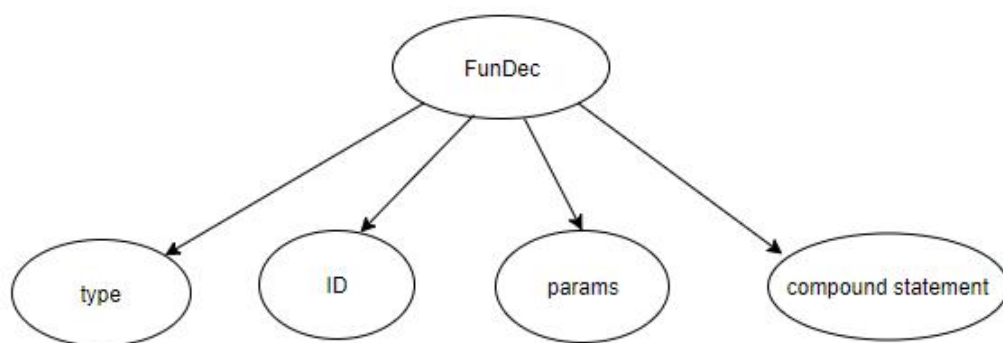


图 5 函数定义结点

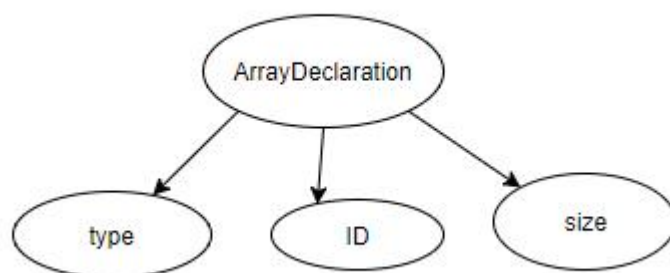


图 6 数组定义结点

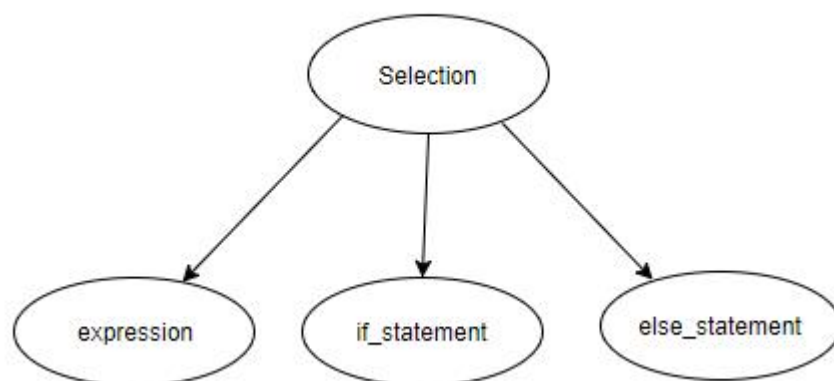


图 7 选择语句结点

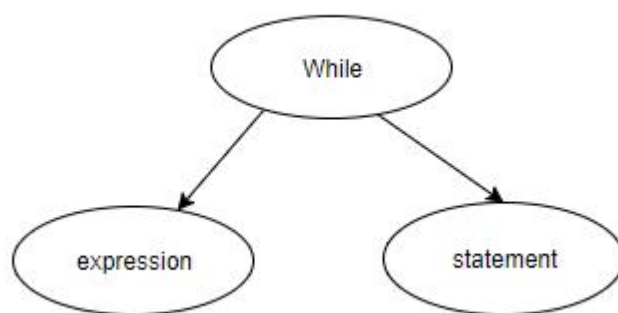


图 8 循环语句结点

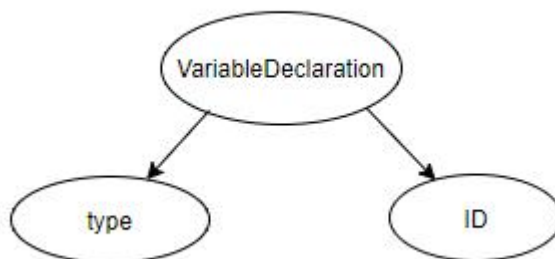


图 9 简单变量定义结点

抽象语法树的代码结构见 globals.h 文件。

2.5 生成抽象语法树 (AST)

flex 使用正则表达式识别词法，将输入内容分解成一个个 token 并返回给 bison 使用。而 bison 则通过用户规定的语法规则将这些 token 按语法逻辑组合起来。

语法分析可以完全交给 bison 使用 LALR (1) 处理，但语义分析需要用户自己定义，即给每条产生式附上相应的语义动作。

设计好抽象语法树的结点之后，给每一条产生式附上一条语义规则，用于生成抽象语法树。(Cminus.y 文件先定义了词法分析的 token，它们和 Cminus.l 文件中的词法分析 token 返回值一一对应，供语义分析使用。)

例子：

fun_declaration

```
: type_specifier _id LPAREN params RPAREN compound_stmt  
  { $$ = newFunctionDeclarationNode($1, $2, $4, $6); }  
;
```

其中\$\$代表左端非终结符的综合属性，在这里它就是函数定义子树的根结点，\$1代表产生式右侧第一个非终结符的综合属性。newFunctionDeclarationNode()的作用是根据已有的信息（函数返回类型子树\$1、函数名子树\$2、参数列表子树\$3、函数体子树\$4）创建一棵函数定义子树，并将其根结点的指针赋予\$\$，从而创建了一棵函数定义子树。其中 LPAREN 和 RPAREN 就是词法分析的 token，即左括号和右括号终结符。

其他产生式的语义规则详见 Cminus.y 文件。完成语义动作的函数详见 util.c 文件。

这样 bison 在语法分析使用某条产生式进行规约时，就执行所定义的语义动作，构建 AST。

使用 bison 时需要注意以下几点：

- ① token 要与 flex 的.l 文件一一对应。
- ② 要用 “%start program” 指定文法的开始符号。

使用 bison 的详细代码在 Cminus.y 中。

2.6 中间代码 (IR) 生成

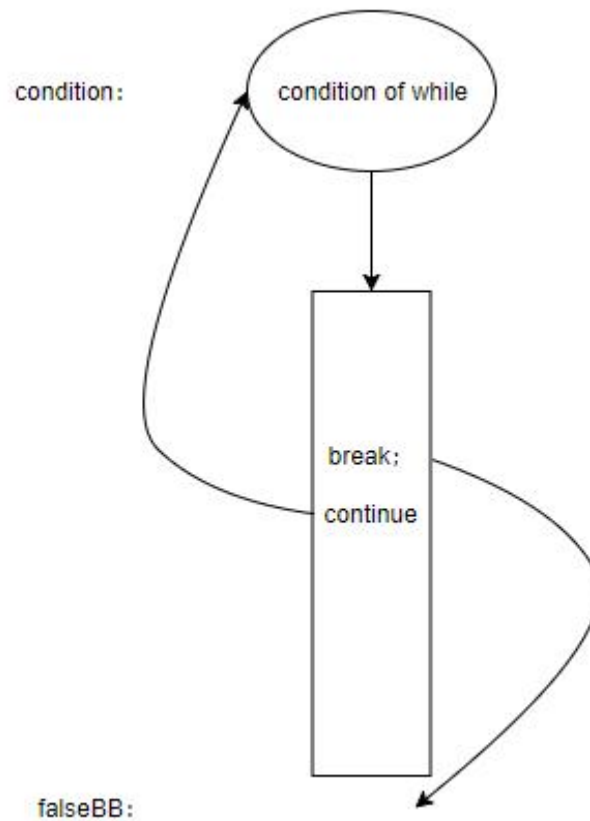
中间代码生成阶段，每一种抽象语法树结点都对应着单独的动作。

在遍历抽象语法树时生成符号表，每一个像这样{ 语句 1; 语句 2; ……，语句 n; }的复合语句对对应一张符号表，维护一个符号表栈就可处理复合语句的嵌套使用。每一张表记录着在本复合语句内定义的变量，栈中所有的其他表都是这张表的祖先，在本复合语句中可以引用其祖先复合语句中定义的变量。

对于选择语句结点，需要三个标签，true 标签，false 标签，false 基本块之后第一条代码位置的标签，在 LLVM 的 IR 中，它们分别对应一个基本块。根据条件产生条件跳转语句，接着生成 true 基本块的代码，false 基本块的代码，并在这俩基本块的末尾无条件跳转至 false 之后的基本块。

循环语句结点同理，只不过基本块最后是无条件跳转至条件判断语句处，其余类似。另外注意到，以 while 循环为例，如果其中有 break 语句，其实只是简单地无条件跳转到与 while 直接对应的复合语句的后面，只需要加一个临时变量记录现在处于哪个 block 中，然后查表找到其直接后继 block 跳转过去即可，continue 则是无条件跳转到其对应的条件判断语句。但是目前时间紧张，没在代

码中实现它们。



加法结点，首先产生获取左、右运算量的代码，再产生一条加法语句，将加法结果挂到 AST 的本结点上，供其父节点使用。其父节点可能是赋值语句、四则运算语句等等。

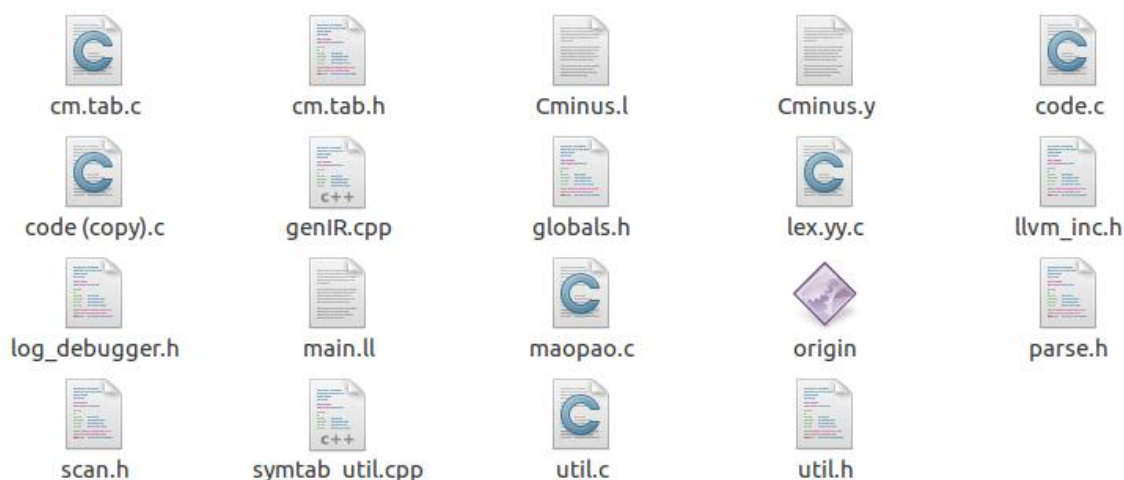
需要注意，LLVM IR 要使用一个变量的值，要先将它从内存中 Load 进来 (比如 Load 到寄存器)，才能使用，不能直接使用主存上的值。

局限：没有实现全局变量的定义与使用，数组也不能作为函数参数。

Talk is cheap.详细代码在 genIR.cpp 文件中，其注释非常详细。

3 系统主要函数说明

系统的所有文件有：



其中 `cm.tab.c`、`cm.tab.h`、`lex.yy.c` 是 flex、bison 根据用户定义的词法 (`cm.l`) 和语法 (`cm.y`) 生成的。`globals.h` 中定义一些全局工具变量，包括 AST 结点结构定义。`llvm_inc.h` 是调用 LLVM API 时需要包含的头文件及其他通用头文件。`scan.h` 和 `parse.h` 中定义词法、语法分析使用的函数的声明。从 AST 到 IR 的主要函数在 `genIR.cpp` 中；在 `symtab_util.cpp` 打印符号表。`util.h` 和 `util.c` 中定义所有构建 AST 结点所需的工具函数，作为语义动作使用。`log_debugger.h` 中有打印各种信息的工具函数；`origin` 是编译以上所有文件所生成的可执行文件。其他是测试用例。

3.1 yylex()

这是 flex、bison 自动生成的函数，调用它会返回词法分析中下一个 token。

3.2 TokenType getToken(void);

在此函数中调用 `yylex()`，从而可增加用户所需的功能，如每获取一个 token，就打印出它的相关信息，如所在行号，token 值等。

3.3 parse()

这是 bison 根据用户自定义的语法和语义规则自动生成的函数，调用它意味着开始进行语法分析，它的返回值是用户赋予文法开始符号的属性，在这里它的返回值就是 AST 的根结点的指针。

3.4 util.c 文件中的工具函数

`util.c` 文件中定义了创建 AST 各种类型结点所需的函数，它们作为产生

式的语义动作使用，每用一条产生式进行规约意味着产生一个新的结点或直接使用已有结点作为产生式左部符号的属性。一个函数对应一种 AST 结点。

3.5 printTab(map<function_name,function_Info> functions)

printTab 函数以所有函数的符号表为参数，输出符号表的信息。

```

1 int add(int a, int b) {
2     return a + b;
3 }
4
5 int main(void) {
6     int a;
7     int b;
8     int sum;
9     a = 10;
10    b = 5;
11    sum = 0;
12    while (a > 0) {
13        if (b > 0) {
14            sum = add(sum, a);
15            b = b - 1;
16        }
17        else {
18            sum = sum + 1;
19        }
20        a = a - 1;
21    }
22    return sum;
23 }

```

```

***** Function Declaratioin Information *****
Function name: add
Function's parameters' infomation :
Decline  Name      Type      ArrayLength  UsedLines
1        a         int              2,
1        b         int              2,

*** Functions are used in this function ***
null
*** This function is used at WHERE ***
Caller   LineNo
null

***** Function Declaratioin Information *****
Function name: main
Function's parameters' infomation :
Decline  Name      Type      ArrayLength  UsedLines
6        a         int              9,12,14,20,20,
7        b         int             10,13,15,15,
8        sum        int             11,14,14,18,18,22,

*** Functions are used in this function ***
a *** This function is used at WHERE ***
Caller   LineNo
null
icap@ubuntu:~/mycode/2019-12-25$

```

3.6 getIR(TreeNode * tree)

传入 AST 的根节点，以获取中间代码 IR，是从 AST 到 IR 的核心函数。对

以上一段代码产生的 IR 为：有点长，位于附件 main.ll 中。用 lli 指令执行这段 IR，得到最终结果为：

```
main.ll
icap@ubuntu:~/mycode/2019-12-25$ lli main.ll
icap@ubuntu:~/mycode/2019-12-25$ echo $?
45
icap@ubuntu:~/mycode/2019-12-25$
```

3.7 yyerror(char * message)

若分析中遇到错误，输出错误信息。在以上一段代码中把变量 a 改名为 a1，因为标识符只能由字母组成，所以报错。

```
icap@ubuntu:~/mycode/2019-12-25$ ./origin
Lexical analyze error at line 6
Current token: a1
Syntax tree:
  (null)
Code source name: code.c
icap@ubuntu:~/mycode/2019-12-25$
```

4. 中间代码优化

此部分由蔡月嘯主要负责，我们阅读大量资料和样例，掌握 pass 的基本过程之后，在 Mac 上搭建了相关环境。LLVM 不愧是 Apple 的产品，一搭就成功了。随后我们实现了 5 个分析类 pass 和 1 个转换类 pass。细节详见蔡月嘯的报告。

5. 课设心得

“柳暗花明疑无路，山重水复又一村”。

[参考资料]

- [1] 老师的指导
- [2] 同学的讨论
- [3] 参考代码中的 `cminus-compiler-master`
- [4] flex 与 bison <https://blog.csdn.net/CrazyHeroZK/article/details/87359818>
- [5] 使用 Flex Bison 和 LLVM 编写自己的编译器
<https://blog.csdn.net/haoel/article/details/4789364>
- [6] LLVM IRBuilder 类函数 http://llvm.org/doxygen/classllvm_1_1IRBuilder.html
- [7] LLVM 安装 <https://blog.csdn.net/chikey/article/details/85004556>
- [8] LLVM 教程 <https://llvm-tutorial-cn.readthedocs.io/en/latest/chapter-1.html>
- [9] 在 LLVM 中编写 pass 的详细教程 系列
<https://blog.csdn.net/baimafujinji/article/details/78631322>
- [10] LLVM 如何写一个 pass <https://www.jianshu.com/p/9f450969121b>
- [11] 程序设计语言编译原理. 陈火旺. 第 3 版. 国防工业出版社
- [9] 还有很多, 以上的最为关键.